

LLM Benchmark Report

Generated: 2026-09-25T10:14:00
Models: Creative Writing, Creative Writing

Performance Summary

Task	Creative Writing latency / tok·s ⁻¹	Creative Writing latency / tok·s ⁻¹
factual	33.56s ~ 1.4 tok/s	6.0s ~ 8.7 tok/s
reasoning	40.73s ~ 4.1 tok/s	5.0s ~ 27.8 tok/s
code	36.57s ~ 4.5 tok/s	9.0s ~ 24.2 tok/s
instruction	11.51s ~ 1.3 tok/s	3.0s ~ 10.0 tok/s
creative	74.07s ~ 0.2 tok/s	4.0s ~ 3.75 tok/s

Aggregate Statistics

Model	Avg Latency (s)	Avg Tok/s	Tasks	Errors
Creative Writing	39.29	2.3	5	0
Creative Writing	5.4	14.9	5	0

Detailed Responses

factual

Prompt: Answer concisely (1-2 sentences each): 1. What is the time complexity of quicksort in the average case? 2. What does HTTP status 429 mean? 3. What year was Python first released? 4. What is the capita

Factual Recall · 33.56s · 47 tokens · ■ Manual

1. The average-case time complexity of quicksort is $O(n \log n)$. 2. HTTP status 429 means "Too Many Requests," indicating the user has sent too many requests in a given amount of time. 3. Python was first released in 1991. 4. The capital of Kazakhstan is Astana (formerly Nur-Sultan).

Factual Recall · 6.0s · 52 tokens · ■ Manual

1. $O(n \log n)$ average case, due to the partition step running $O(n)$ across $O(\log n)$ average recursion depth. 2. Too Many Requests – the client has exceeded the server's rate limit. 3. 1991. 4. Astana (renamed from Nur-Sultan in 2022).

reasoning

Prompt: A bat and a ball cost \$1.10 in total. The bat costs \$1.00 more than the ball. How much does the ball cost? Show your reasoning step by step.

Logical Reasoning · 40.73s · 168 tokens · ■ Manual

To find the cost of the ball, we can use a simple algebraic approach. ### Step-by-Step Reasoning: 1. **Define the variables:** - Let the cost of the ball = b - The bat costs \$1.00 more than the ball, so: bat = $b + 1.00$ 2. **Set up the equation:** - Total cost: $b + (b + 1.00) = 1.10$ 3. **Solve:** - $2b + 1.00 = 1.10$ - $2b = 0.10$ - $b = 0.05$ ### Answer: The ball costs **\$0.05 (5 cents)**. *Note: The common intuitive answer of \$0.10 is incorrect. If the ball were \$0.10, the bat would be \$1.10, totaling \$1.20 – not \$1.10.*

Logical Reasoning · 5.0s · 139 tokens · ■ Manual

Let the ball cost x dollars. The bat costs \$1.00 more than the ball, so the bat costs $(x + 1.00)$ dollars. Together they cost \$1.10: $x + (x + 1.00) = 1.10$ $2x + 1.00 = 1.10$ $2x = 0.10$ $x = 0.05$ The ball costs \$0.05 (5 cents). The bat costs \$1.05, and the difference is exactly \$1.00. Total: \$1.10. ✓ The intuitive answer of 10 cents is wrong – if the ball were 10 cents, the bat would be \$1.10, and the total would be \$1.20, not \$1.10.

code

Prompt: Write a Python function `find_duplicates(lst)` that returns a list of all elements that appear more than once, in the order they first appear. Include a docstring and 3 assert statements that test it.

Code Generation · 36.57s · 165 tokens · ■ Manual

```
python def find_duplicates(lst): """ Returns a list of all elements that appear more than once in `lst`, in the order they first appear. Args: lst: An iterable of hashable elements. Returns: A list of duplicate elements, preserving first-occurrence order. """ seen = set() duplicates = [] seen_duplicates = set() for item in lst: if item in seen: if item not in seen_duplicates: duplicates.append(item) seen_duplicates.add(item) else: seen.add(item) return duplicates assert find_duplicates([1, 2, 3, 2, 1, 4]) == [2, 1] assert find_duplicates(['a', 'b', 'a', 'c', 'b']) == ['a', 'b'] assert find_duplicates([1, 2, 3]) == []
```

Code Generation · 9.0s · 218 tokens · ■ Manual

```
```python def find_duplicates(lst): """ Return a list of elements that appear more than once in lst, in the order of their first occurrence. Args: lst: A list of hashable elements. Returns: A list of duplicate elements, preserving first-occurrence order. """ seen = set() duplicates = [] seen_duplicates = set() for item in lst: if item in seen and item not in seen_duplicates: duplicates.append(item) seen_duplicates.add(item) seen.add(item) return duplicates assert find_duplicates([1, 2, 3, 2, 1, 4]) == [2, 1] assert find_duplicates(["a", "b", "a", "c", "b", "b"]) == ["a", "b"] assert find_duplicates([1, 2, 3]) == [] ``` Two sets do the work in a single pass – seen tracks everything encou...
```

---

## instruction

*Prompt: List exactly 5 sorting algorithms. Format as a numbered list. For each: name only, no description. Do not include any other text before or after the list.*

**Instruction Following** · 11.51s · 15 tokens · ■ Manual

```
1. Bubble Sort 2. Selection Sort 3. Insertion Sort 4. Quick Sort 5. Merge Sort
```

**Instruction Following** · 3.0s · 30 tokens · ■ Manual

```
1. Bubble Sort 2. Merge Sort 3. Quick Sort 4. Heap Sort 5. Insertion Sort
```

---

## creative

*Prompt: Write a haiku about running AI models locally on your own hardware. Follow the 5-7-5 syllable structure strictly.*

**Creative Writing** · 74.07s · 12 tokens · ■ Manual

```
Chips hum, models run, Thinking on my own machine, My own silicon.
```

**Creative Writing** · 4.0s · 15 tokens · ■ Manual

```
Silicon hums low Models think without the cloud Mind lives in the box
```